

Cost-Aware Repository Change Localization with Sparse Impact

Planning and Bounded Verification

MSc Research Proposal (V1.2, supervisor-ready / print-candidate)

Ahmed Ehab

2026-09-16

Abstract

Large language models (LLMs) that edit a software repository must first decide which files a requested change affects. This decision is costly: an explicit file-by-file impact policy over a repository-scale candidate universe spends hundreds of serialized records per call, and a second-stage reasoner that revisits the scope can multiply inference cost by orders of magnitude. This thesis asks a single central question:

Can a cheap/sparse first-pass repository localization policy followed by bounded omission recovery achieve a better correctness–cost trade-off than cheap localization alone and always-on expensive reasoning?

We investigate a representation that is cheap by construction — *preserve-by-omission* sparse impact plans, which emit only non-preserve decisions — and evaluate it together with cheap lexical/static baselines and an explicit *bounded* second-stage omission-recovery mechanism, all under matched inference budgets. Preliminary controlled and real-commit studies show a large serialization-cost reduction for the sparse representation and a meaningful zero-LLM lexical signal, but also that task-level omission risk is not reliably predictable on current development evidence. The thesis therefore evaluates the representation and the correctness–cost Pareto frontier, and tests a pre-registered candidate-level bounded-verification mechanism, rather than requiring a task-level risk scorer to succeed.

1 Background, motivation, and problem

Repository-level coding agents and impact planners must select affected files before editing. Two problems motivate the thesis. First, *impact inference*: did the model identify the right files? Second, *impact-policy representation*: can a complete file-action policy be expressed without spending output budget on hundreds of repeated *preserve* decisions? Our controlled study shows that an explicit full policy at a 16K completion cap serializes on average 144 decision records per candidate universe, while a sparse policy that emits only non-preserve decisions serializes on average 4.9 records (a $\sim 96.6\%$ reduction), with large associated reductions in completion tokens and cost.

Second-stage reasoning is expensive. A graph-guided agent evaluated under a shared protocol consumed on the order of a million tokens per task on the same held-out tasks where our sparse first pass stays below a few thousand tokens. If a cheap first pass already localizes the bulk of affected files, the question is whether a *bounded* second look can recover missed files without paying the full always-on cost.

Given a repository revision P (parent commit), a natural-language change request q , and a candidate universe $U(P)$ of production files visible at P , a localization system produces an affected-file set $\hat{S}(P, q)$. The hidden observed change set $S^*(P)$ is used for evaluation only. The

thesis studies correctness–cost trade-offs under explicit budgets: representation cost (sparse vs explicit) and bounded second-stage verification cost.

The observed historical diff is treated as an *observed change-set proxy*, never as semantic ground truth (Section 6).

2 Central thesis hypothesis and gap

Hypothesis. A sparse/cheap first-pass localization policy followed by a bounded omission-recovery mechanism can dominate both cheap-localization-only and always-on expensive reasoning on the correctness–cost Pareto frontier, at least for the repositories and protocols we evaluate.

Everything else in the thesis supports this central story:

- *Preserve-by-Omission* = representation / first-pass contribution;
- *BM25 / classical static change-impact analysis* = cheap baselines;
- *graph-guided agent* = expensive system-level context;
- *task-level omission-risk scorer* = investigated and currently rejected on development evidence;
- *candidate-level bounded verification* = primary next mechanism;
- *second and third repositories* = generalization.

Task-level RiskScorer success is *not* a thesis requirement. Existing work covers several individual components — agentic graph-guided localization, repository memory / retrieval-conditioned generation, explicit edit planning, agentless localize-then-repair, and classical change-impact analysis. This thesis evaluates *their combination* as a sparse first pass plus bounded omission recovery under matched budgets (see Section 4).

3 Research questions and expected contributions

RQ1. How does sparse explicit impact-policy representation affect inference/output cost and file-selection fidelity?

RQ2. How accurately can cheap lexical/static methods localize impacted files relative to LLM planners under shared protocols?

RQ3. Can bounded second-stage verification recover missed impacted files more efficiently than always-on or random matched-budget verification?

RQ4. How do these correctness–cost trade-offs transfer across repositories and ecosystems?

If task-level risk modeling remains unsupported by evidence, RQ3 is answered by the candidate-level mechanism; *RiskScorer success is not required*.

Expected contributions:

1. Sparse explicit impact-plan representation with controlled evidence.
2. A reproducible real-commit impact-localization dataset and protocol.
3. Fair cheap/classical/LLM/agent comparison under common file-level metrics and explicit budgets.

4. A bounded/selective-compute verification protocol (candidate-level by default; task-level only if evidence justifies it).
5. Cross-repository correctness–cost evidence.
6. Reproducibility artifacts and leakage-control methodology.

No positive algorithmic result is promised.

4 Positioning against related work

Existing work covers several individual components. We avoid any claim of the form “none combines...” unless a systematic matrix supports it. Our positioning: this thesis evaluates the *combination* of (a) an explicit sparse impact-policy representation, (b) a cheap first pass, and (c) a bounded omission-recovery mechanism, all under matched budgets, on real commits across repositories.

Verified related lines (primary sources, 2026-09-16) and their relation:

- *Cost-aware LLM change-impact analysis*: requirements-level CIA under a cost lens (Etezadi et al., arXiv 2511.00262) — same cost-aware question, requirements-to-requirements unit, not file localization.
- *History / change-pattern augmentation of CIA*: Change-Patterns Mapping (TSE 2022), history-learned impact tools (ICSE-SEIP 2022), learning dependency-based predictors (IST 2015) — the classical/ history lineage that our Route B history evidence must be positioned against.
- *Modern transform + program-dependence-graph impact analysis*: Yan et al. (arXiv 2607.23355) — graph+LLM impact analysis is active; our graph is one optional verifier.
- *Selective-compute / efficiency+verification priors*: Repoformer (ICML 2024) and FastCoder (arXiv 2502.17139) — selective retrieval and efficient verification at repository level (code-completion/generation tasks), guarding any generic “selective escalation is novel” claim.
- *Graph-guided issue localization*: LocAgent, GraphLocator (arXiv 2512.22469), and LLM-driven iterative code-graph search (arXiv 2503.22424) — the expensive graph-guided competitors.
- *Recommendation-system CIA with industrial grounding*: Borg et al. (TSE 2017) — classical lineage and industrial validity.

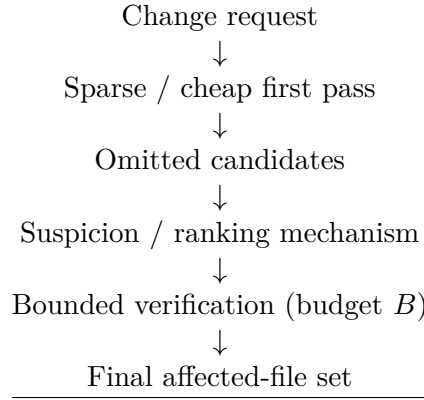
The comparison matrix (Section 7) records for each line whether it has an explicit file policy, sparse omission representation, cheap first pass, omission recovery, hard/matched budget, real-commit evaluation, and cross-repository evidence. Where novelty is only defensible once experiments support it, it is marked *candidate novelty* — *not yet claimed*.

5 Proposed two-stage architecture

1. **Sparse/cheap first-pass impact policy**: explicit file-level policy with preserve-by-omission serialization; cheap deterministic and LLM planner variants under the same contract.

2. **Candidate-level bounded verification** (primary next mechanism): from the first-pass omitted candidates, rank suspicious candidates with independent structural / retrieval / history evidence, and verify only those under a hard per-task budget B . Compare with always-verify and random matched-budget verification.

The flow:



6 Dataset construction, labels, and leakage controls

- **Real history, parent-only inputs.** Inference uses the parent commit P ; the target T is hidden. Live repository HEAD is never used (it moves and can leak future state).
- **Eligibility and exclusions.** Candidates must be single-parent commits with a meaningful natural-language intent, touching only production source files. Tests, configurations, documentation, migrations, and generated/vendor files are excluded from the candidate universe because they are not the target of file-level impact edits in this protocol. Leakage barrier: a case whose intent mentions a changed path is ineligible.
- **Deduplication.** Exact-proxy-set and shared-reference duplicates and suspected-related changes are removed with frozen deterministic rules, so accepted cases are independent.
- **Observed change-set proxy.** $P \rightarrow T$ changed production files are an *observed* proxy, never semantic impact gold. Localization claims are scoped to historical-file recovery unless semantic adjudication (Section 10.2) supports more.
- **Splits.** A metadata-only, deterministic split (seed, hashes) is frozen before any model result. For djangoCMS V2-LARGE: development, an untouched internal test, and a reserve. The v1 40 cases are LEGACY-EXPOSED and never reused as untouched confirmation.

7 Experimental design, baselines, budgets, metrics, statistics

7.1 Experimental Design at a Glance

7.2 Baselines

Random/cheap control, BM25, adaptive cheap retrieval (if justified), classical/static change-impact analysis, Sparse planner, Full planner, graph-guided reference agent (when reproducible), Sparse + Always Verify, Sparse + Random Verify (matched budget), Sparse + Selective/Bounded Verify, and an Oracle-routing upper bound. If task-level routing is unsupported, the selective arm becomes candidate-level bounded verification.

Study	Purpose	Repo	Split	Methods	Unit	Reps	Budget	Metrics	Status
Representation	sparse-vs-full cost	curated	controlled	Full vs Sparse	task	3	cap 16K	tokens, records, cost, P/R/F1	done
Real-commit	selection fidelity	djangoCM5	held-out	Full vs Sparse	task	3	cap 16K	P/R/F1/FNR	done
Graph agent	expensive text	djangoCM5	held-out	agent	task	1	per-call	P/R/F1, cost	done
Cheap baselines	zero-LLM sig-nal	djangoCM8	dev	BM25/graph	task	–	cheap	P/R/F1	done
Task risk	risk predictability	djangoCM8	dev	features	task	3	–	AUROC/AUPRC	done (neg.)
Candidate verify	omission recovery	djangoCM8	dev	rankers	candidate	–	$B=5$	FN-recovery	proposed
Cross-repo	transfer	Saleor/NesTBD	FBD	frozen method	task	–	frozen	per-repo	proposed

Table 1: Experimental Design at a Glance (internal IDs appear in traceability notes only).

7.3 Related-work comparison matrix

Line	File pol.	Sparse rep.	Cheap 1st	Omission recov.	Matched budget	Real commit	X-r
Classical CIA (ripple/change-propagation)	no	no	yes	partial	no	some	n
IR localization	no	no	yes	no	no	some	n
RepoCoder / RepoAgent	no	no	no	no	no	some	n
CodePlan	yes	no	no	no	no	no	n
Agentless	no	no	partial	no	no	SWE-bench	n
LocAgent	no	no	no	no	no	yes*	n
GraphLocator	no	no	no	no	no	yes	n
Repoformer / FastCoder	no	no	partial	no	selective	no	n
This thesis	yes	yes	yes	yes	yes	yes	y

* LocAgent is evaluated here under a shared protocol (P5); not a faithful reproduction of the published fine-tuned result.

7.4 Frozen budget definition

For zero-LLM candidate-ranking (Route B):

- **Primary verification budget:** B = number of omitted candidate files admitted to second-stage inspection.
- **Primary operating point:** $B = 5$ candidates per task.
- **Secondary:** $B \in \{1, 3, 10\}$.
- **Primary endpoint:** task-clustered FN recovery / missed-file recall gain at $B = 5$.
- **Key control:** Sparse + Random Verify @ $B = 5$.
- **Secondary:** revised FNR/F1, task coverage, scope inflation, runtime.

For a future LLM verifier: primary compute measure = additional total model tokens per task; secondary operational measures = calls, USD cost, latency, failures. Tokens, dollars, and latency are never collapsed into one metric.

7.5 Multiple-comparison discipline

One primary endpoint at $B=5$; one primary Selective-vs-Random comparison; secondary endpoints clearly labeled; task-level paired bootstrap; no feature-family winner chosen by dozens of post-hoc tests; exploratory analyses reported as exploratory.

7.6 Statistics

Task is the independent unit; nested repetitions are never treated as independent observations. Pre-registered class-balance gate for any risk modeling: ≥ 10 independent positives and ≥ 10 negatives, a predeclared feature above the random band, and direction stability across development / validation. Bootstrap confidence intervals over tasks; random-risk bands at the observed n ; sample-size planning uses development evidence only.

8 Preliminary evidence

All numbers are from audited studies in this repository; they are development or controlled evidence and are reported with their bounds.

- **Representation effect (controlled).** At a 16K completion cap, Sparse serialized on average **4.9** decision records vs **144.0** for Full (a **~96.6%** reduction), with mean completion tokens ~ 809 vs 8,383 and a large recorded-cost reduction. No universal semantic-superiority claim is made.
- **Historical real-commit study.** 60 cells over 10 held-out real djangoCMS commits (Full vs Sparse, 3 nested repetitions): selection is difficult, FNR is high, and there is no Full-vs-Sparse superiority claim (paired deltas' confidence intervals cross zero). Sparse execution is substantially cheaper.
- **Graph-guided agent comparison.** System-level shared-protocol context only: 402 calls / 32.8M tokens / $\sim \$9.93$ estimated on 10 tasks, with 5/10 non-usable outcomes (2 true timeouts + 1 context-length error + 2 completed-but-empty). Not a reproduction of the published fine-tuned result.
- **Cheap/static development baselines.** BM25 provides a meaningful zero-LLM localization signal on development data; graph-ranked seeds are comparable; K is an operating-point curve, not a fixed configuration. A classical change-impact (dependency propagation) baseline is implemented on 150 development cases.
- **Task-level omission-risk investigation.** Live Sparse development inference (90 cells; then 431 calls on 144 new development tasks with a fail-closed stop at the nominal 2.5M-token ceiling — the final completed call produced a 1,964-token, $\sim 0.08\%$ post-call overshoot, and the run stopped before another call). A task-level RiskScorer is *not* justified: the v1 candidate signal does not replicate within the new development cohort, and the apparent pooled signal is a candidate-universe-size artifact. No multivariable RiskScorer is fitted.

These results motivate a larger development dataset, protection of an untouched internal test, and the bounded-verification design; they are not rewritten as a success.

9 Cross-repository generalization

Stage 1 djangoCMS V2-LARGE (within-repo large evidence + untouched internal test) $\rightarrow$ Stage 2 Saleor (different real system, same Python/Django stack) $\rightarrow$ Stage 3 NestJS (cross-language / cross-framework evidence).

Each stage states its claim boundary: within-repo development + internal confirmation; cross-repository confirmation; cross-ecosystem evidence. NestJS is preferred but not forced; a predeclared eligible-pool yield criterion and a backup protocol are frozen.

10 Threats, contingency, and timeline

10.1 Threats to validity

- **Internal:** the observed proxy is not semantic gold; leakage control (parent-only inputs, intent-path-leakage off) is mandatory; no hidden-test peeking.
- **External:** n and single-repository generalization; V2 and cross-repository evidence are the mitigation, with per-repository primary metrics.
- **Statistical:** low negatives in the development label; multiple comparisons; wide random bands — addressed by pre-registration, class-balance gates, and bootstrap uncertainty.
- **Representation:** LLM serialization cost spans providers; we report both recorded and frozen-pricing cost and keep the provider pinned as far as allowed.

10.2 Semantic-proxy validity (preparation)

Because the historical diff is only a proxy, we prepare a stratified *development-only* sample (20–30 tasks) with evidence packets (public intent, parent context, $P \rightarrow T$ diff, changed production files, candidate universe metadata) for later human adjudication. Human questions: was each changed production file semantically necessary; is there evidence of semantically impacted but unchanged files; is the commit mixed with incidental refactor.

Machine-assisted notes are prepared but are never called semantic gold. Protocol:

`docs/SEMANTIC_PROXY_AUDIT_PROTOCOL.md`.

10.3 Contingency

If candidate-level bounded verification does not beat random at the primary budget, the negative result is frozen and reported; no complex verifier is invented.

10.4 Timeline

Window	Activity
2026-09/10	Proposal V1.2; V2-LARGE freeze + build
2026-10	V2 development inference; cheap + classical baselines; candidate-level verification
2026-11	Bounded-verification experiments; Saleor protocol + inference
2026-12	NestJS cross-ecosystem check; Pareto analysis
2027-01/02	Analysis, replication, threat revisions
2027-03/04	Write-up; internal-test confirmation
2027-05	Submission-ready thesis

11 References

The authoritative bibliography is `references.bib` (V1.2). References were verified against primary sources (arXiv API) on 2026-09-16. Preprints are marked. No fabricated or placeholder citations remain.